

Spring Boot 入门篇

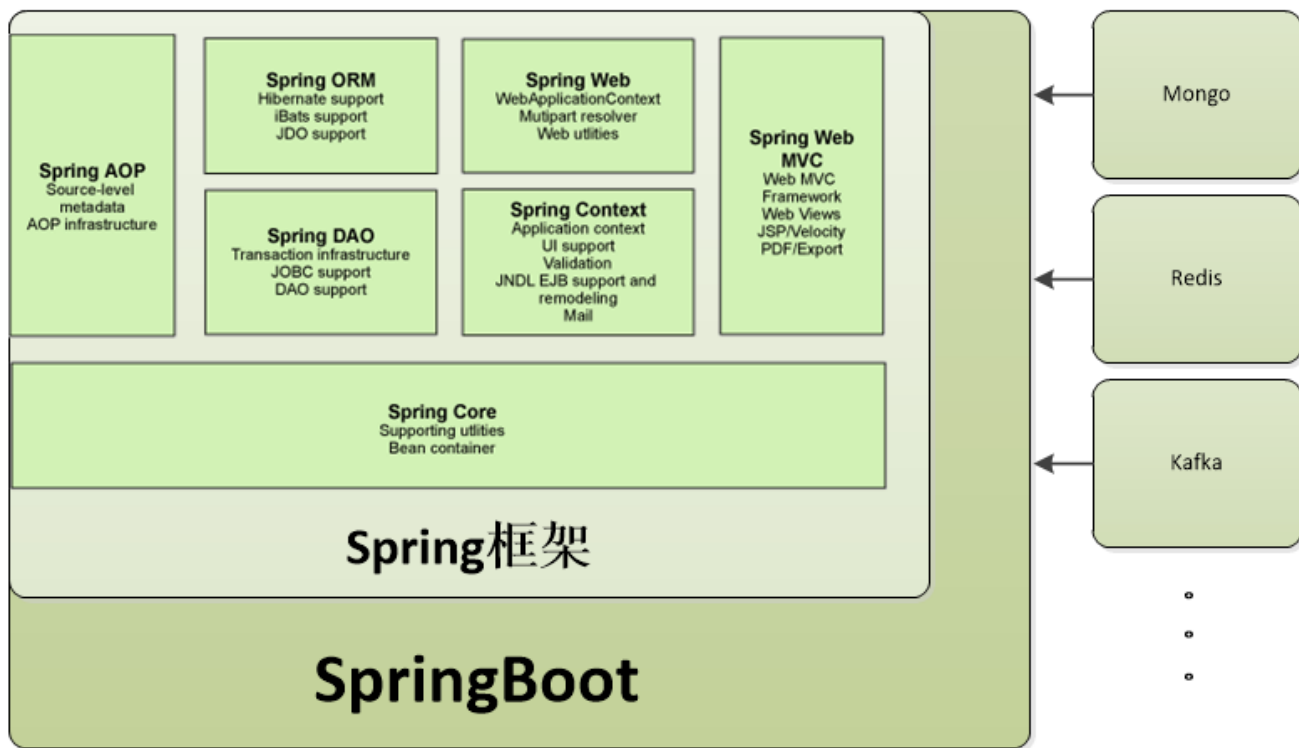
王天青 | 2024年7月

目录 >

CONTENTS

- 1 快速入门
- 2 Web 开发
- 3 数据访问
- 4 缓存使用
- 5 Spring Security
- 6 可观测性
- 7 Spring Boot Starter

Spring Boot 简介



- Spring 框架是一个“家族”，有众多衍生产品例如 boot、security、JPA 等等
- Spring 的配置非常复杂，各种XML、JavaConfig 处理起来比较繁琐。
- Spring Boot，约定优于配置，简化了spring的配置流程。

“Spring Boot 是一个基于 Java 的开源框架，用于创建微服务，构建独立的生产就绪的 Spring 云原生应用。”

Spring Boot 特点

- **独立运行的 Spring 项目**

Spring Boot 可以以 jar 包的形式独立运行，Spring Boot 项目只需通过命令 “`java -jar xx.jar`” 即可运行。

- **内嵌 Servlet 容器**

Spring Boot 使用嵌入式的 Servlet 容器（例如 Tomcat、Jetty 或者 Undertow 等），应用无需打成 WAR 包。

- **提供 starter 简化 Maven 配置**

Spring Boot 提供了一系列的 “starter” 项目对象模型（POMS）来简化 Maven 配置。

- **提供了大量的自动配置**

Spring Boot 提供了大量的默认自动配置，来简化项目的开发，开发人员也通过配置文件修改默认配置。

- **自带应用监控**

Spring Boot 可以对正在运行的项目提供监控。

- **无代码生成和 xml 配置**

Spring Boot 不需要任何 xml 配置即可实现 Spring 的所有配置。

1. Spring Boot - Hello World(RESTful API)



Project

☒ Maven Project

☐ Gradle Project

Language

☒ Java ☐ Kotlin

☐ Groovy

Spring Boot

☐ 2.6.0 (SNAPSHOT) ☐ 2.6.0 (RC1) ☐ 2.5.7 (SNAPSHOT)

☐ 2.5.6 ☐ 2.4.13 (SNAPSHOT) ☒ 2.4.12

Project Metadata

Group com.example

Artifact demo

Name demo

Description Demo project for Spring Boot

Package name com.example.demo

Packaging ☒ Jar ☐ War

Java ☐ 17 ☒ 11 ☐ 8

Dependencies

ADD DEPENDENCIES... % + B

Spring Reactive Web

WEB

Build reactive web applications with Spring WebFlux and Netty.

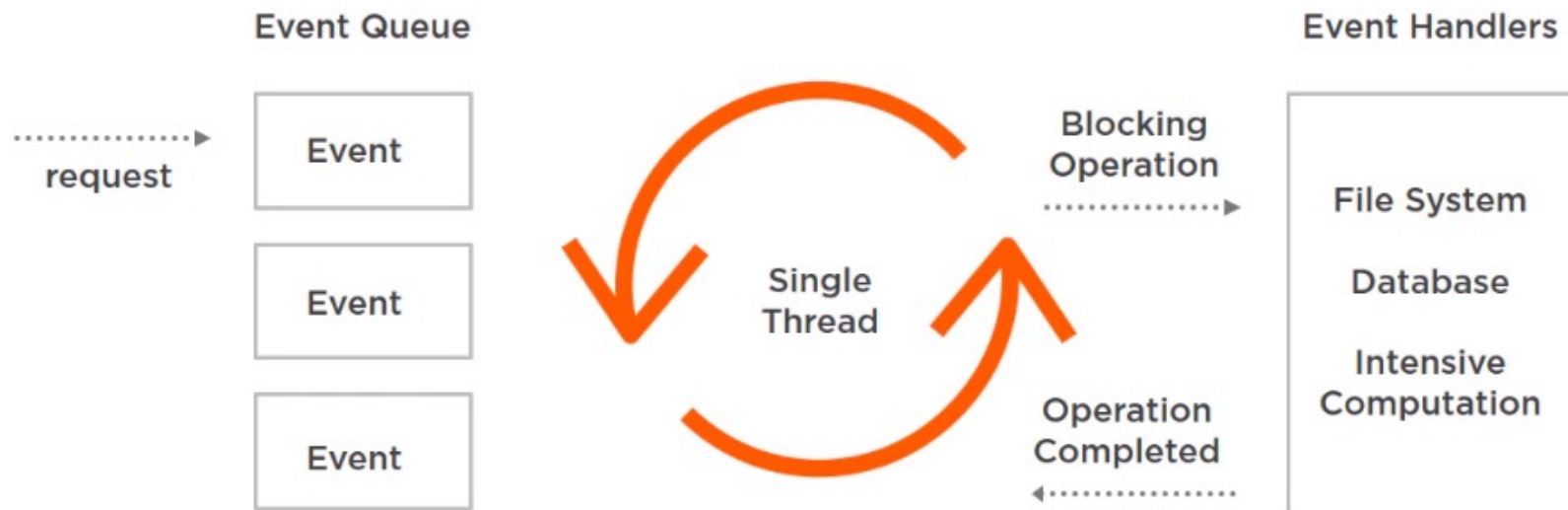
The screenshot shows an IDE with the following components:

- Project Structure:** A tree view on the left showing the project hierarchy: demo > src > main > java > com > example > demo > DemoApplication. Other folders like .idea, .mvn, and test are also visible.
- Code Editor:** The main window displays the `DemoApplication.java` file. The code is as follows:

```
1 package com.example.demo;
2
3 import ...
4
5 @EnableWebFlux
6 @SpringBootApplication
7 public class DemoApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(DemoApplication.class, args);
11     }
12 }
```
- File Explorer:** A sidebar on the right shows the file explorer with the following files: DemoApplication, HelloController, pom.xml, application.properties, resources, application.properties, test, java, com.example.demo, DemoApplicationTest, target, .gitignore, demo.iml, and HELP.md.

代码演示: HelloController

Spring WebFlux 简介

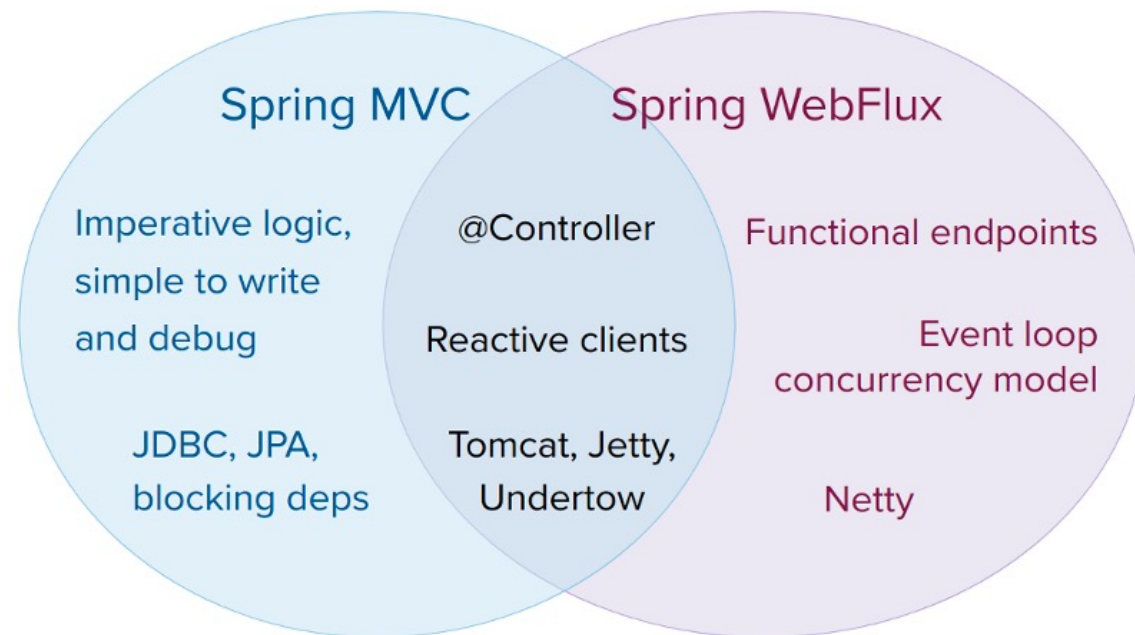
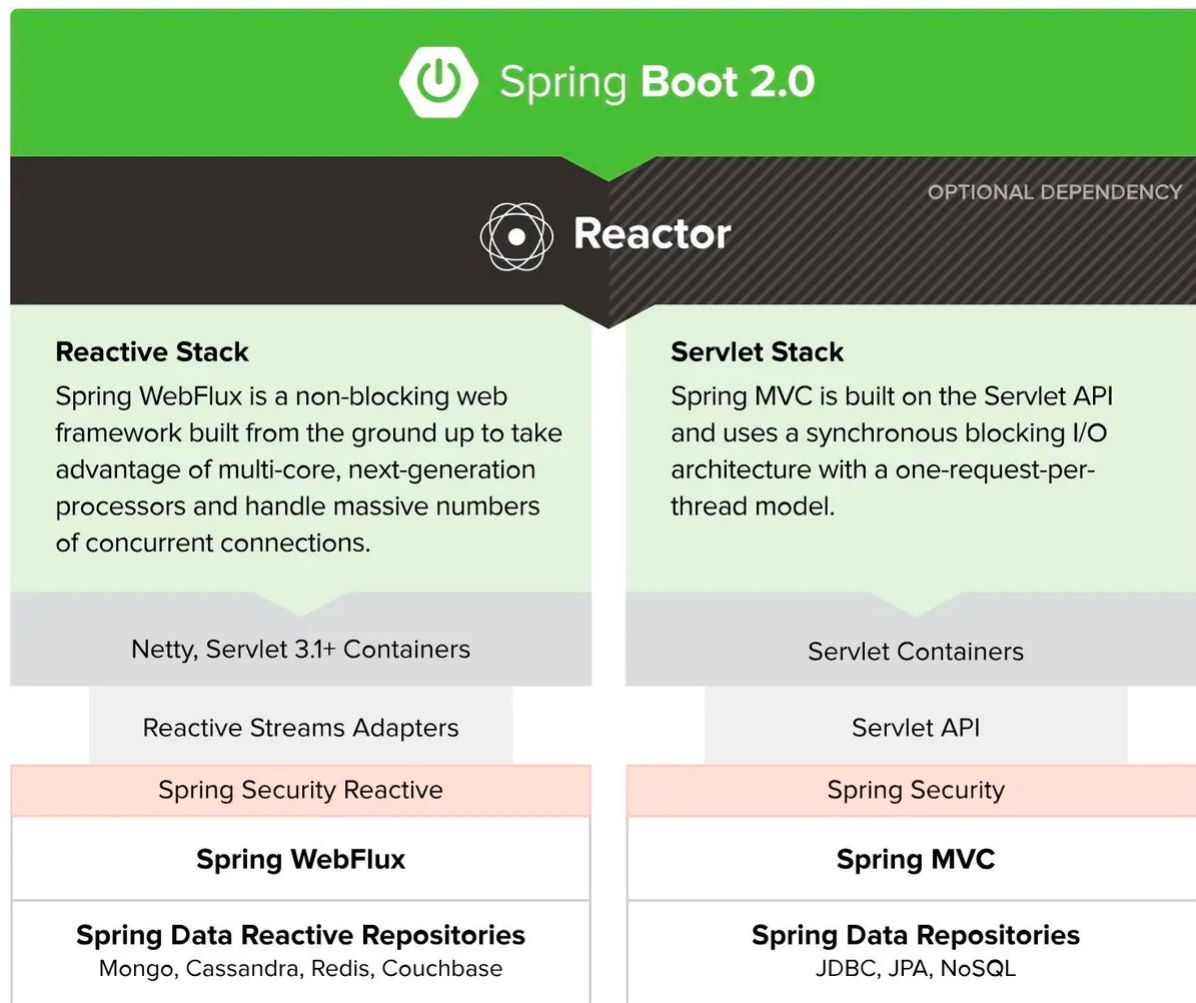


Spring 5.0 在原有的 Spring MVC Stack（又称Servlet Stack）以外，又引入了新的WEB开发技术栈——Spring Flux Stack（又称 Reactive Stack），以满足不同的应用程序及开发团队的需求。

WebFlux是Spring 5提供的响应式Web应用框架。

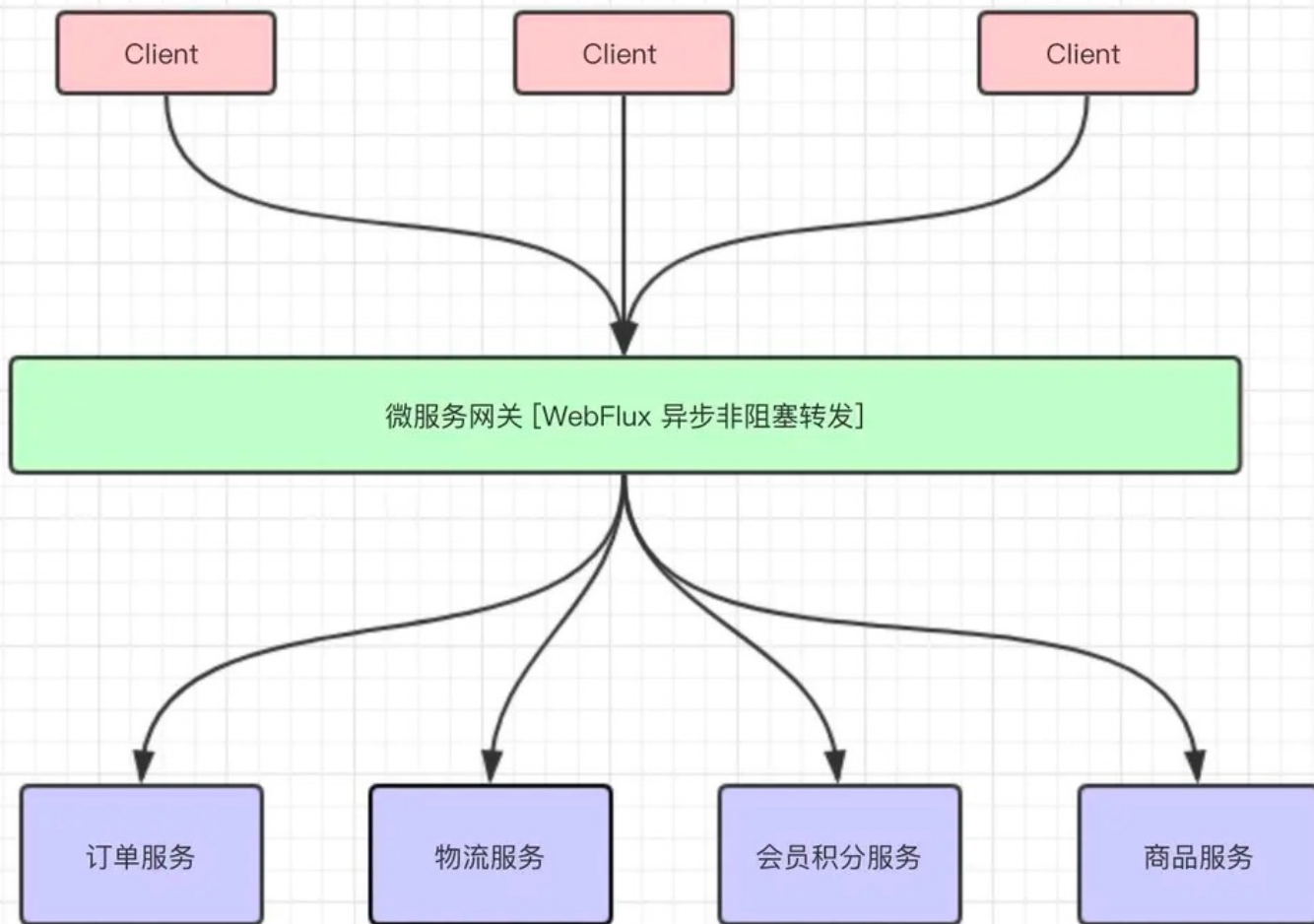
它是完全非阻塞的，可以在Netty，Undertow和Servlet 3.1+等非阻塞服务器上运行。

Spring MVC 与 WebFlux 异同点



代码演示: **UserRouter**

WebFlux 应用场景



它特别适合应用在 IO 密集型的服务中，
IO 密集型包括：磁盘IO密集型, 网络IO密集型，
微服务网关就属于网络 IO 密集型，
使用异步非阻塞式编程模型，
能够显著地提升网关对下游服务转发的吞吐量。

JPA 顾名思义就是 Java Persistence API 的意思，是 JDK 5.0 注解或 XML 描述对象 - 关系表的映射关系，并将运行期的实体对象持久化到数据库中。

特点：
基于注解的使用

注解	解释		
@Entity	声明类为实体或表。		
@Table	声明表名。		
@Basic	指定非约束明确的各个字段。	@ManyToMany	定义了连接表之间的多对多一对多的关系。
@Embedded	指定类或它的值是一个可嵌入的类的实例的实体的属性。	@ManyToOne	定义了连接表之间的多对一的关系。
@Id	指定的类的属性，用于识别（一个表中的主键）。	@OneToMany	定义了连接表之间存在一个一对多的关系。
@GeneratedValue	指定如何标识属性可以被初始化，例如自动、手动、或从序列表中获得的值。	@OneToOne	定义了连接表之间有一个一对一的关系。
@Transient	指定的属性，它是不持久的，即：该值永远不会存储在数据库中。	@NamedQueries	指定命名查询的列表。
@Column	指定持久属性栏属性。	@NamedQuery	指定使用静态名称的查询。
@SequenceGenerator	指定在@GeneratedValue注解中指定的属性的值。它创建了一个序列。		
@TableGenerator	指定持久属性栏属性。		
@Column	指定在@GeneratedValue批注指定属性的值发生器。它创造了的值生成的表。		
@AccessType	指定持久属性栏属性。		

- 引入 Spring Data JPA 和 H2_(内存数据库) 依赖
- application.properties 配置数据源
- 创建 Entity
- 创建 JPA Repository
- Service 使用 Repository
- Controller 使用 Service
- Configuration/AppConfig 配置初始化 Demo 数据

代码演示: CustomerController

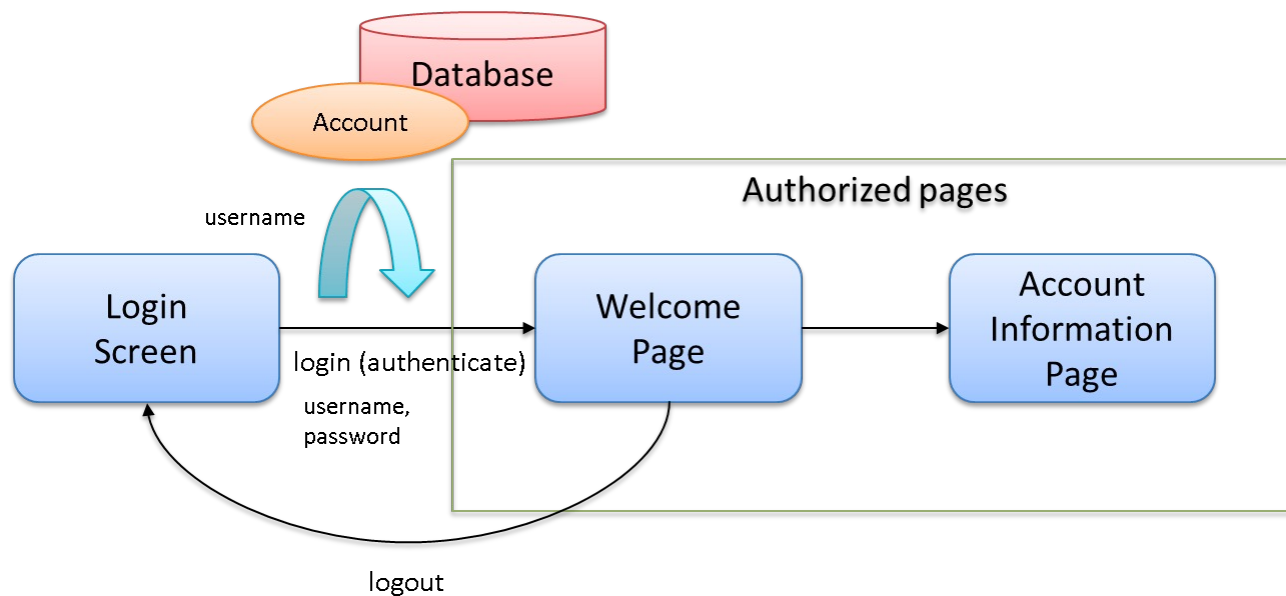
- 将前面的数据库 (Customer) 数据进行缓存
- 引入 Spring Data Redis 和 Spring Boot Cache 依赖
- 准备 Redis: `docker run -d -p 6379:6379 redis`
- `Configuration/CacheConfig` 配置缓存策略
- `application.properties` 配置数据源:
 - `spring.redis.host=localhost`
 - `spring.redis.port=6379`
- 为 Service 添加 `@Cacheable` 注解即可使用缓存功能

代码 API 演示: `/customer/{id}`

Spring Security 是一种基于 Spring AOP 和 Servlet 过滤器 Filter 的安全框架，它提供了全面的安全解决方案，提供在 Web 请求和方法调用级别的用户鉴权和权限控制。

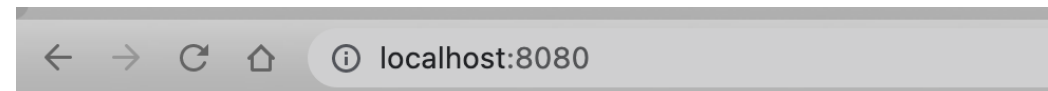
Web 应用的安全性通常包括两方面：用户认证（Authentication）和用户授权（Authorization）。

Spring Security 功能的实现主要是靠一系列的过滤器链相互配合来完成的。



我们这里模拟一个登陆页面，当用户输入内置的用户名和密码并且认证成功过后，才能放我们其他 API 的资源。

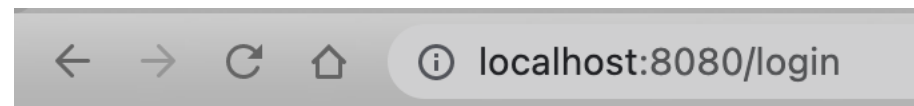
- Spring Boot Security依赖
- 创建用户页面 (login.html, home.html, hello.html)
- 配置 MvcConfig，将视图公开出去
- 配置 WebSecurityConfig：
 - 确保只有经过身份验证的用户才能看到其他信息
- 运行程序，访问 localhost:8080 进入主页



Welcome!

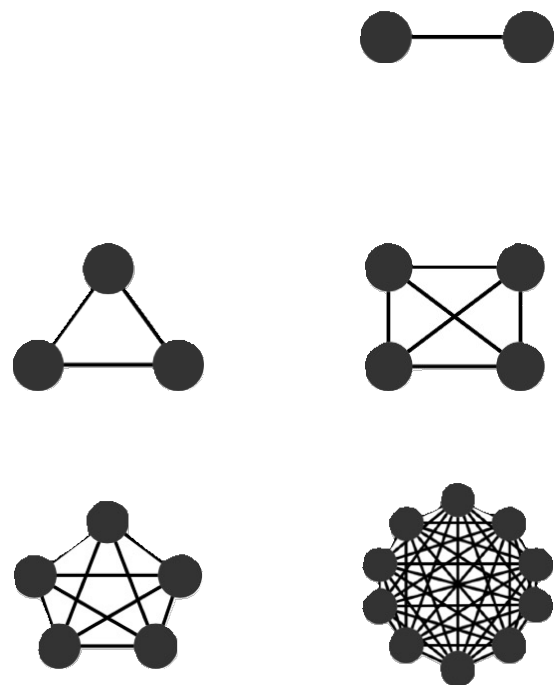
Click [here](#) to see a greeting.

Click [here](#) to find all cutomers.

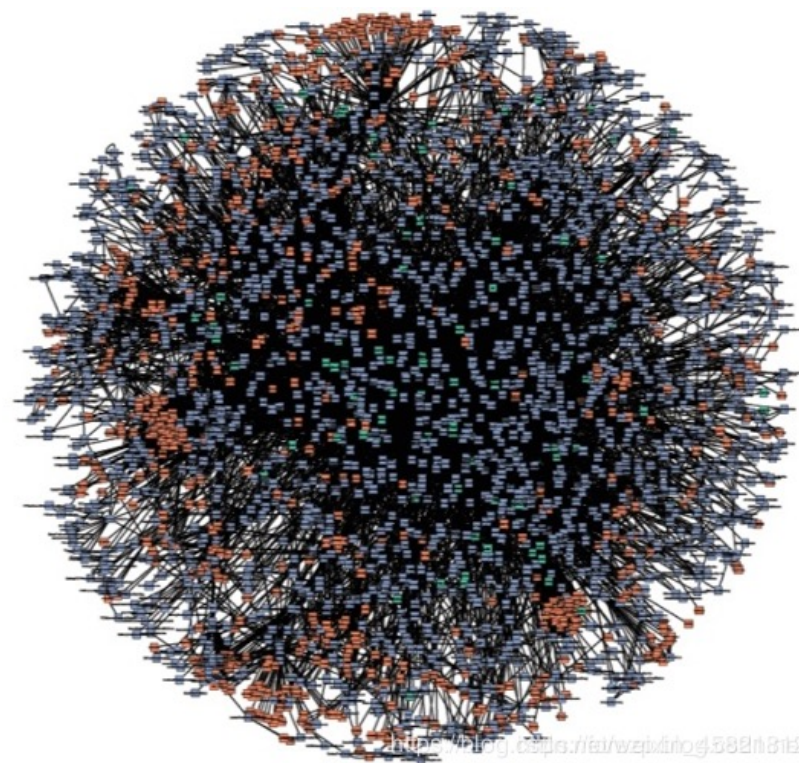


User Name :

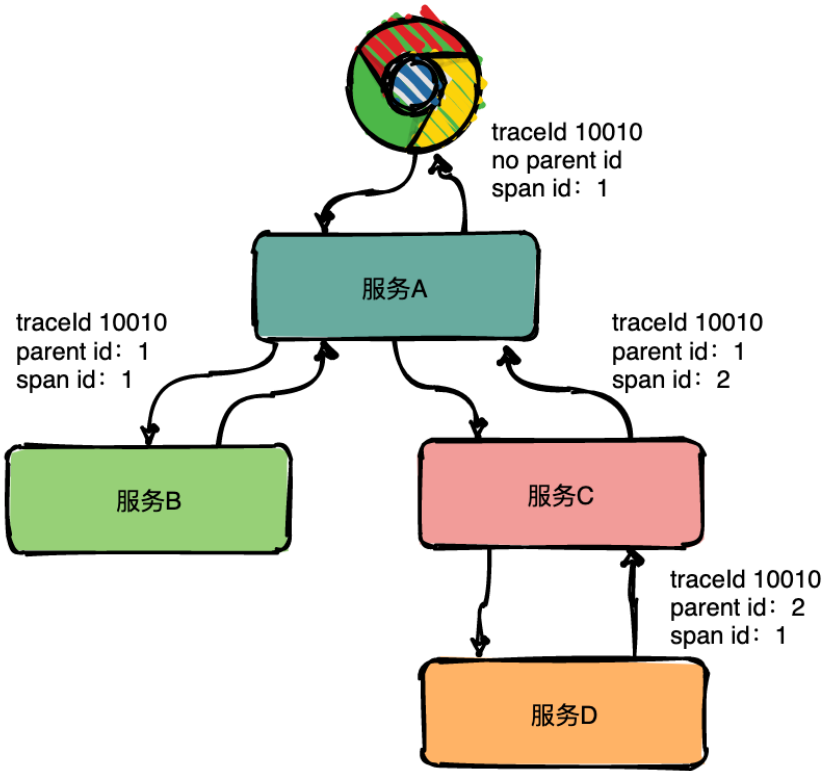
Password:



微服务增多，关系复杂



链路串联与可视化



Starters 可以理解为启动器，它包含了一系列可以集成到应用里面的依赖包，你可以一站式集成 Spring 及其他技术，而不需要到处找示例代码和依赖包。如你想使用 Spring JPA 访问数据库，只要加入 `spring-boot-starter-data-jpa` 启动器依赖就能使用了。

传统的做法:

在没有starter之前，假如我想要在 Spring 中使用 jpa，那我可能需要做以下操作：

- 在Maven中引入使用的数据库的依赖（即JDBC的jar）
- 引入jpa的依赖
- 在xxx.xml中配置一些属性信息
- 反复的调试直到可以正常运行

Spring Boot Starter 包的设计

统一的版本管理：spring-boot-starter-parent 这个特殊的 starter 包来统一管理所有其他 starter 包的版本。项目中不需要在 maven 的 jar 包管理文件 pom.xml 为 starter 包指定版本号，只需要指定该 starter 包的 groupId 和 artifactId 即可。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org
<modelVersion>4.0.0</modelVersion>
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.4.12</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- jpa -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>
```

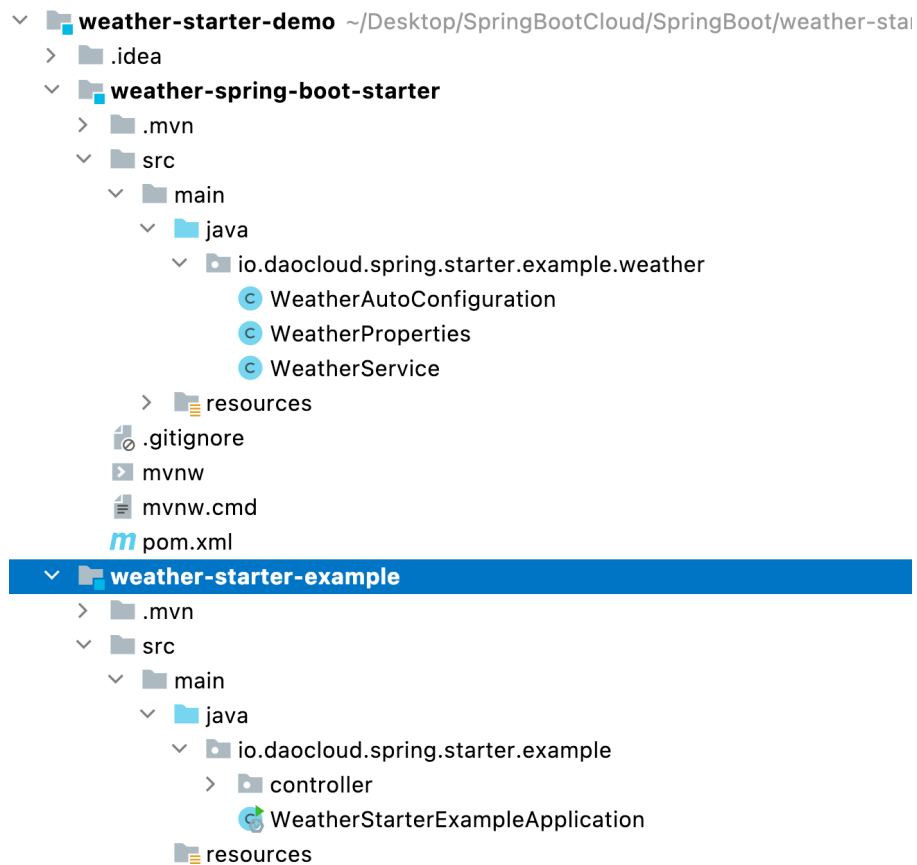
自定义 Starter: 编写一个查询天气的 starter 包

将项目中的公用代码，如被多个模块引用的某个功能，抽离成独立的 jar 包来供多个项目复用。

Spring Boot 官方建议我们在自定义 starter 时，创建两个 Module：autoConfigure Module 和 starter Module，其中 starter Module 依赖于 autoConfigure Module。

这里演示将它们组合到同一个 Module 里。

参考：[weather-starter-demo项目](#)



自定义 Starter 一：定义 propertie 类

```
import org.springframework.boot.context.properties.ConfigurationProperties;

/**
 * 用来映射查询天气的配置信息
 */
// 将该类中的属性与配置文件中的以 demo.weather 开头的配置进行绑定。
@ConfigurationProperties("demo.weather")
public class WeatherProperties {
    //demo.weather.queryApi
    private String queryApi;

    //demo.weather.apiType
    private String apiType;

    public String getQueryApi() { return queryApi; }

    public void setQueryApi(final String queryApi) { this.queryApi = queryApi; }

    public String getApiType() { return apiType; }

    public void setApiType(final String apiType) { this.apiType = apiType; }
}
```

将该类中的属性与配置文件中的以 demo.weather 开头的配置进行绑定。

自定义 Starter 二：定义 Service 类

```
@Service
public class WeatherService {

    WeatherProperties weatherProperties;

    public WeatherService(final WeatherProperties weatherProperties) {
        this.weatherProperties = weatherProperties;
    }

    /**
     * 根据给定的地区代号，查询天气情况 代号查询: https://www.cnblogs.com/wangjingblogs/p/3192953.html
     *
     * @param areaCode
     * @return
     */
    public String query(String areaCode) {
        String url = weatherProperties.getQueryApi() + areaCode + weatherProperties.getApiType();
        RestTemplate restTemplate = new RestTemplate();
        HttpHeaders headers = new HttpHeaders();
        HttpEntity<String> entity = new HttpEntity<>(headers);
        String strbody = restTemplate.exchange(url, HttpMethod.GET, entity, String.class).getBody();
        try {
            // 解决中文乱码问题
            return new String(strbody.getBytes( charsetName: "ISO-8859-1"), charsetName: "UTF-8");
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

查询接口可以在使用 Starter 的时候自定义，不配置则使用内置的

```
demo.weather.queryApi=http://www.weather.com.cn/data/cityinfo/
demo.weather.apiType=.html
```

自定义 Starter 三：定义配置类，自动注入

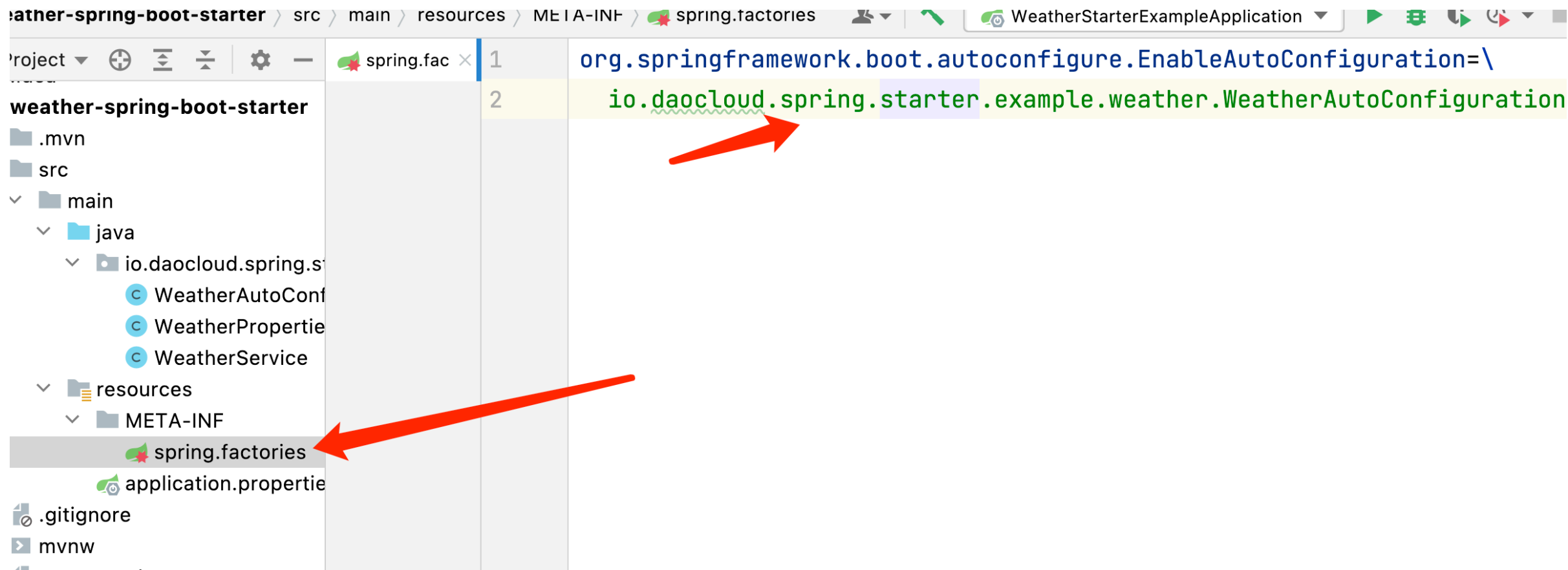
```
@Configuration
@EnableConfigurationProperties(WeatherProperties.class) //启用 WeatherProperties, 并默认将
public class WeatherAutoConfiguration {
    @ConditionalOnMissingBean(WeatherService.class) //当容器中没有 WeatherService 时生效

    @Bean
    public WeatherProperties weatherProperties() {
        return new WeatherProperties();
    }

    //将方法的返回值以 Bean 对象的形式添加到容器中。
    @Bean
    public WeatherService weatherService(WeatherProperties weatherProperties) {
        return new WeatherService(weatherProperties);
    }
}
```

自定义 Starter 四：创建 spring.factories文件

由于 Spring Boot 的自动配置是基于 Spring Factories (SPI) 机制实现的，因此我们自定义 starter 时，同样需要在项目类路径下创建一个 spring.factories 文件。



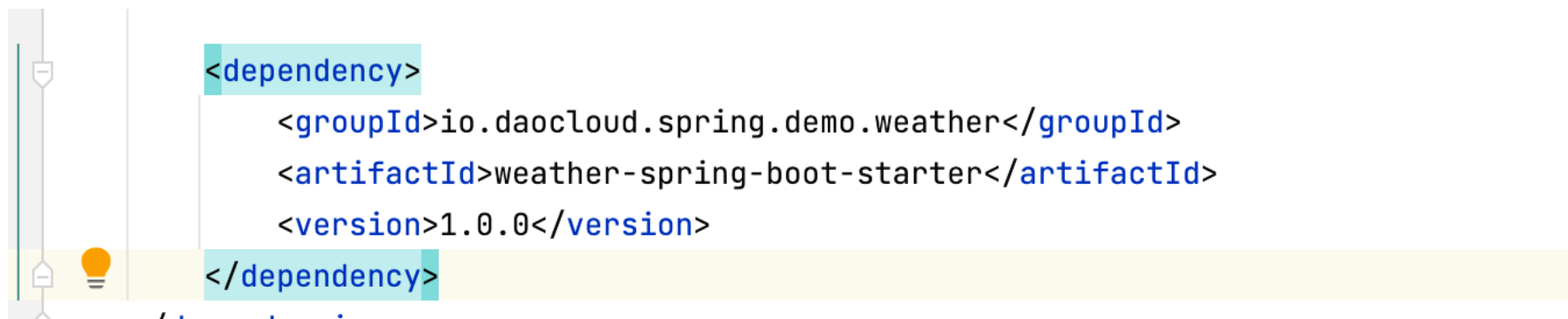
自定义 Starter 五：构建/发布 Starter

mvn clean install 将 Starter 安装到本地 Maven 仓库，方便调试。
正式发布供他人使用时则可以用 mvn clean deploy 发布远程 Maven/Nexus（私有）仓库

```
[INFO]
[INFO] --- maven-install-plugin:2.5.2:install (default-install) @ weather-spring-boot-starter ---
[INFO] Installing /Users/tanjian/Desktop/SpringBootCloud/SpringBoot/weather-starter-demo/weather-spring-boot-starter/target/tanjian/.m2/repository/io/daocloud/spring/demo/weather/weather-spring-boot-starter/1.0.0/weather-spring-boot-starter-1.0.0.pom
[INFO] Installing /Users/tanjian/Desktop/SpringBootCloud/SpringBoot/weather-starter-demo/weather-spring-boot-starter/pom.xml to /Users/tanjian/Desktop/SpringBootCloud/SpringBoot/weather-starter-demo/weather-spring-boot-starter/1.0.0/weather-spring-boot-starter-1.0.0.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.510 s
[INFO] Finished at: 2022-03-12T16:17:59+08:00
[INFO] -----
```

使用自定义的 Starter

参考: [weather-starter-example](#)



启动 weather-starter-example 并访问该服务:

```
curl localhost:8080/weather/query?areaCode=101010100
```

```
curl localhost:8080/weather/query?areaCode=101020100
```

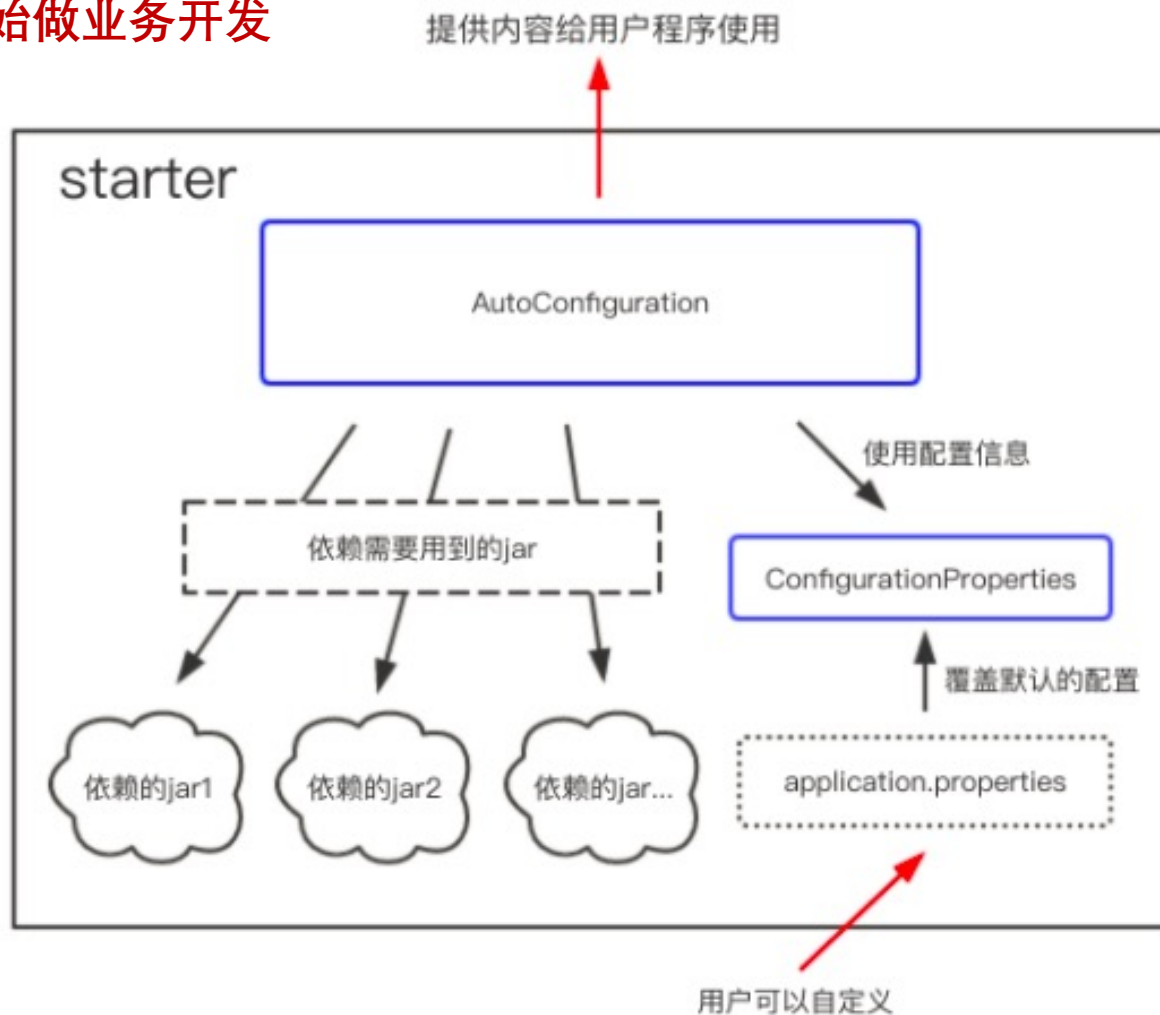
```
{"weatherinfo":{"city":"北京","cityid":"101010100","temp1":"18°C","temp2":"31°C","weather":"多云转阴",  
,"img1":"n1.gif","img2":"d2.gif","ptime":"18:00"}}%
```

Spring Boot - 数据库访问/JPA

- 引入 Spring Data JPA 和 H2(内存数据库) 依赖
- application.properties 配置数据源
- 创建 Entity
- 创建 JPA Repository
- Service 使用 Repository
- Controller 使用 Service
- Configuration/AppConfig 配置初始化 Demo 数据

代码演示: CustomerController

引入依赖就可以开始做业务开发





Q&A

TRANSWARP
星 环 科 技